

UVM-Based Verification of AXI-to-APB Bus Protocol

Verification Project Report

Submitted by:

Muhammad Hammad
Zain-ul-Aabideen
Mirza Muhammad Umar

Supervisor:

Engr. Abdur-Rafy

1. Introduction

This project verifies a bridge that connects an AXI master to an APB slave, using a UVM (Universal Verification Methodology) testbench. AXI and APB are both part of ARM's AMBA bus family, but they are built for different jobs: AXI is a high-performance, burst-capable protocol used between fast masters (CPUs, DMA controllers) and memory, while APB is a simple, low-power protocol used to talk to peripheral registers. The bridge's job is to convert AXI transactions into the equivalent APB transactions correctly, and the goal of this verification effort is to prove that it does so under normal operation, corner cases, and error conditions.

2. AXI Protocol Overview

2.1 What Is AXI

- AXI stands for Advanced eXtensible Interface.
- It is part of ARM's AMBA (Advanced Microcontroller Bus Architecture) family.
- It is built around independent, unidirectional address and data channels, which allow multiple transactions to be in flight at once and give it much higher throughput than simpler bus protocols.

AXI is defined in three main variants: AXI4, the full protocol with burst support; AXI4-Lite, a simplified subset with no burst capability, generally used for register-style access; and AXI4-Stream, which drops the address phase entirely and is used for high-speed, unidirectional data streaming. This project targets the full AXI4 protocol on the master side of the bridge.

2.2 AXI Channels

An AXI master and slave communicate over five independent channels. Because each channel operates on its own valid/ready handshake, address and data can move in parallel rather than waiting on a single shared bus.

- 1) Read address channel
- 2) Read data channel
- 3) Write address channel
- 4) Write data channel
- 5) Write response channel

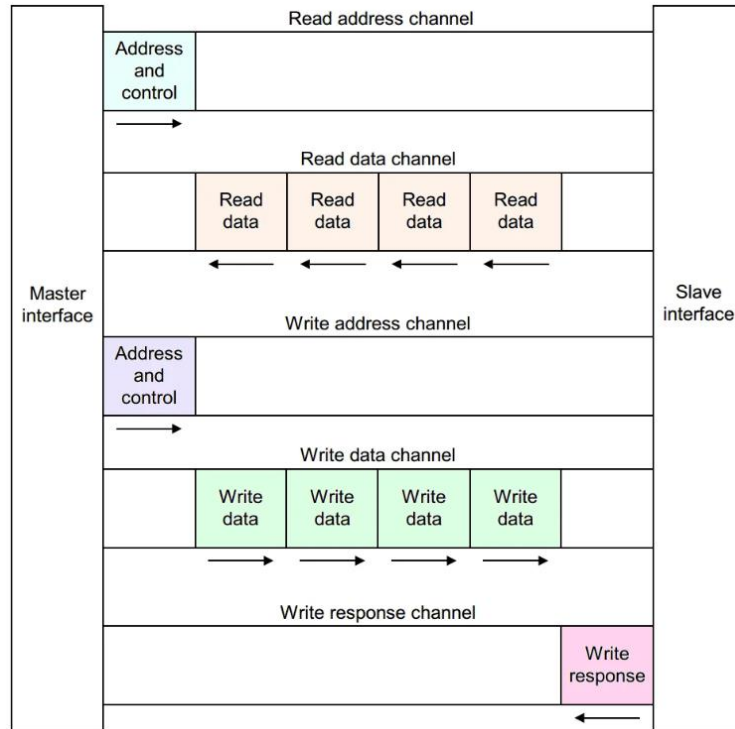


Figure 1: The five AXI channels between master and slave.

2.3 AXI Channel Signals

Each channel carries its own set of address, data and control signals, alongside the VALID/READY handshake pair that governs when a transfer actually takes place. A transfer on any channel only occurs on a clock edge where both VALID and READY are high.

Read Address Channel Signals	Global Signals	Write Address Channel Signals	Write Data Channel Signals
ARID[3:0] Master Read address ID	ACLK Clock source Global clock signal	AWID[3:0] Master Write address ID	WID[3:0] Master Write ID tag
ARADDR[31:0] Master Read address	ARESETn Reset source Global reset signal	AWADDR[31:0] Master Write address	WDATA[31:0] Master Write data
ARLEN[3:0] Master Burst length		AWLEN[3:0] Master Burst length	WSTRB[3:0] Master Write strobes
ARSIZE[2:0] Master Burst size	Read Data Channel Signals	AWSIZE[2:0] Master Burst size	WLAST Master Write last
ARBURST[1:0] Master Burst type	RID[3:0] Slave Read ID tag	AWBURST[1:0] Master Burst type	WVALID Master Write valid
ARLOCK[1:0] Master Lock type	RDATA[31:0] Slave Read data	AWLOCK[1:0] Master Lock type	WREADY Slave Write ready
ARCACHE[3:0] Master Cache type	RRESP[1:0] Slave Read response	AWCACHE[3:0] Master Cache type	Write Response Channel Signals
ARPROT[2:0] Master Protection type	RLAST Slave Read last	AWPROT[2:0] Master Protection type	BID[3:0] Slave Response ID
ARVALID Master Read address valid	RVALID Slave Read valid	AWVALID Master Write address valid	BRESP[1:0] Slave Write response
ARREADY Slave Read address ready	RREADY Master Read ready	AWREADY Slave Write address ready	BVALID Slave Write response valid
			BREADY Master Response ready

Figure 2: Representative AXI channel signal listing.

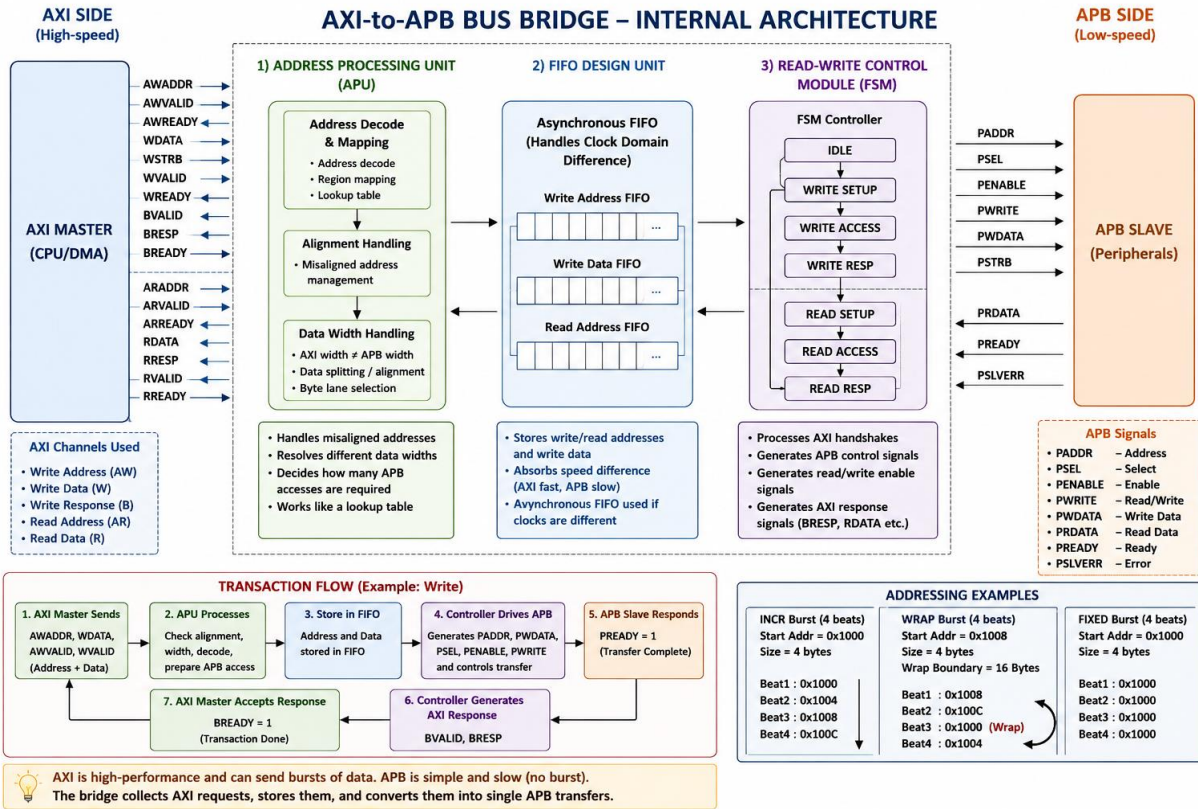


Figure 3: AXI channel signal overview.

2.4 Burst Operation

In an AXI transaction, each individual data transfer is called a beat, and a burst is a group of consecutive beats carried out under a single address phase. Bursting lets a master move several beats of data while only paying the cost of one address handshake, which is a major part of why AXI outperforms simpler bus protocols.

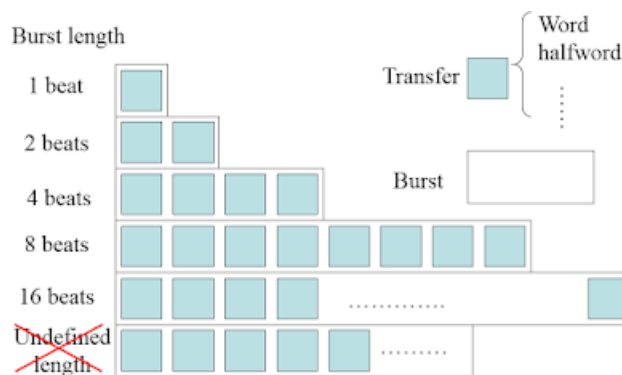


Figure 4: A burst made up of multiple beats.

2.5 Burst Transfer Types

AXI defines three burst types, selected by the AxBURST signal:

- **Fixed burst:** the address stays the same for every beat. Used for repeated accesses to one location, such as filling or draining a peripheral FIFO.
- **Incrementing burst:** the address increases after every beat, by an amount equal to the transfer size (e.g. +4 for a 4-byte transfer).
- **Wrapping burst:** like an incrementing burst, but the address wraps back to a lower boundary once it reaches the wrap limit. The wrap boundary equals transfer size $\times$ burst length, the start address must be aligned to the transfer size, and the burst length must be 2, 4, 8 or 16 beats.

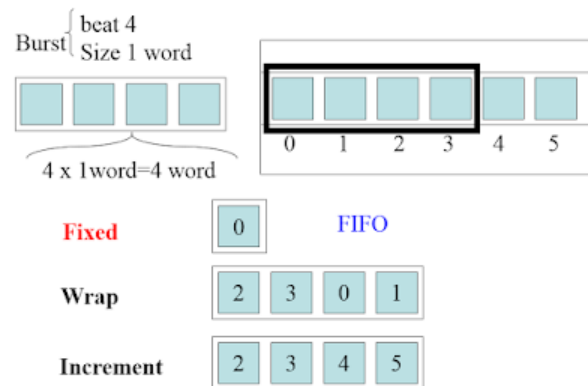


Figure 5: Burst Type

3. APB Protocol Overview

APB (Advanced Peripheral Bus) is the low-complexity, low-power member of the AMBA family. Unlike AXI, it has no burst support, no pipelining, and only a single, non-multiplexed set of address/data/control signals. Every transfer is a simple two-cycle handshake using PSEL, PENABLE and PREADY, which makes APB cheap to implement in peripherals such as UARTs, GPIO and simple control/status registers, but far too slow for high-bandwidth traffic. Because AXI and APB operate so differently, an AXI master cannot connect directly to an APB slave — a bridge is required in between.

4. AXI-to-APB Bridge (Design Under Test)

The bridge is the DUT for this project. It sits between the AXI master and the APB slave and is responsible for:

- Accepting AXI read/write transactions, including bursts, on its AXI slave interface.
- Breaking each burst down into the individual single-beat transfers that APB requires, since APB has no concept of a burst.
- Driving the correct PSEL/PENABLE/PWRITE/PADDR/PWDATA sequence on the APB interface and waiting on PREADY for peripherals that insert wait states.
- Returning read data, write responses, and any APB error (PSLVERR) back to the AXI master on the correct response channel.

5. UVM Verification Methodology

UVM (Universal Verification Methodology) is a standardized, SystemVerilog class library for building reusable, constrained-random testbenches. Rather than writing a fresh testbench for every test, a UVM environment is built once from a set of standard components and then reused across many tests:

- Sequence / Sequencer - generates the stream of AXI transactions (directed or randomized) that drive each test.
- Driver - converts each transaction into pin-level activity on the AXI interface.
- Monitor - passively observes the AXI and APB interfaces and packages the activity into transactions for checking.
- Scoreboard - compares what the bridge actually did on APB against what was expected from the original AXI transaction, and flags mismatches.
- Agent / Environment - bundle the driver, sequencer and monitor together, and combine the AXI-side and APB-side agents into the complete testbench.

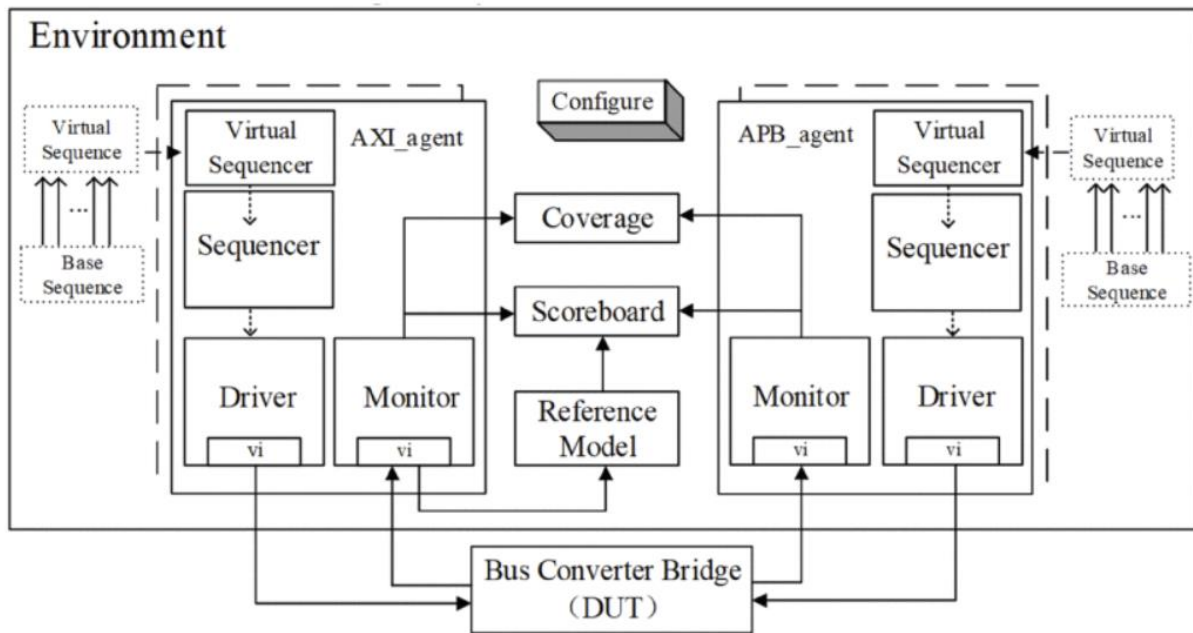


Figure 6: Environment of UVC's

This structure is what makes the test plan below practical: the same environment is reused for simple directed tests, corner cases like wait states and errors, and the large random stress test, without rewriting the testbench each time.

6. Verification Test Plan

The table below summarizes the tests used to exercise the bridge, from basic single-transfer checks through to a long random stress run.

Test Name	AXI Sequence	APB Slave Behavior	What Is Verified
Write Test	Single write	Normal	AXI write transaction correctly translated to an APB write
Read Test	Single read	Returns data	APB read data correctly returned on the AXI read data channel
Random Test	Random reads/writes	Normal	General functional correctness under mixed traffic
Burst Test	Burst write/read	Normal	Burst-to-single-transfer conversion (full AXI only)
Wait-State Test	Read/write	Delayed PREADY	Bridge correctly waits on APB handshake
Error Test	Read/write	PSLVERR = 1	APB error is correctly propagated back to AXI
Reset Test	Transactions after reset	Normal	Bridge recovers cleanly after reset
Stress Test	Thousands of random transactions	Random delays/errors	Overall robustness of the bridge

7. Conclusion

Verifying an AXI-to-APB bridge requires a clear understanding of both protocols AXI's channel structure and burst behavior, and APB's simpler handshake - since the bridge's entire job is to translate correctly between the two. Building the checks around a UVM environment makes it possible to reuse one testbench across directed tests, wait-state and error corner cases, and large-scale random stress testing, giving reasonable confidence that the bridge behaves correctly across normal operation and edge cases alike.